Backend

Technology

The backend is a Flask app using:

- `Flask`
- `Flask-CORS`
- `Flask-Caching`
- `Flask-Compress`
- `pandas`
- `numpy`
- `gunicorn`

Dependencies are listed in `backend/requirements.txt`.

App Entry Point

The API app is `backend/app.py`.

Local debug mode can be started directly from that file, but deployment uses Gunicorn:

```
cd backend
gunicorn app:app
```

The Docker startup script binds Gunicorn to the platform-provided port:

```
gunicorn app:app --bind 0.0.0.0:${PORT:-5000}
```

API Endpoints

`GET /api/players`

Query params:

- `division`, default `1_2`

Returns a sorted JSON array of player names from the division CSV.

`GET /api/teams`

Query params:

- `division`, default `1_2`

Returns a sorted JSON array of team names from the division CSV.

`GET /api/tracks`

Query params:

- `division`, default `1_2`

Returns a sorted JSON array of track names from the division CSV.

`GET /api/player`

Query params:

- `name`, required
- `division`, default `1_2`

Returns:

```
{
  "player": "example",
  "results": ["Track - 9.5 pts (3 races)"]
}
```

This endpoint returns the selected player's best tracks.

`GET /api/player-avg`

Query params:

- `name`, required
- `division`, default `1_2`

Returns:

```
{
  "avg": 92.4,
  "player_name": "example",
  "team_name": "ABC",
  "races": 48
}
```

The overall player average multiplies race score by 12 in `stats.findplayeravg`, so it is presented as a 12-race war-style average rather than raw points per race.

`GET /api/top-team-players`

Query params:

- `team`, required by behavior but not explicitly validated before helper call
- `min_races`, default `12`

- `division`, default `1_2`

Returns formatted strings:

```
["Player - 92.4 pts (48 races)"]
```

`GET /api/top-team-tracks`

Query params:

- `team`, required by behavior but not explicitly validated before helper call
- `division`, default `1_2`

Returns formatted strings:

```
["Track - 62.5 pts (4 races)"]
```

`GET /api/top-tracks`

Query params:

- `track`, required
- `min_races`, default `2`
- `division`, default `1_2`

Returns top players on a track as formatted strings.

`GET /api/top-teams-on-track`

Query params:

- `track`, required
- `min_races`, default `2`
- `division`, default `1_2`

Returns top teams on a track as formatted strings.

Helper Modules

`find.py`

`find.py` provides CSV lookup helpers:

- `findtracklist(csvfile)`
- `findplayernames(csvfile)`
- `findteamnames(csvfile)`

Each helper returns lowercase strings. The API list endpoints do not use these helpers; they read CSVs directly so they can return original capitalization.

`stats.py`

`stats.py` contains the actual analytics calculations:

- `findplayeravg`
- `findteamavg`
- `findtopteamtracks`
- `findtopplayertracks`
- `findtopteamplayers`
- `findtoptracks`
- `findtopteamsontrack`

Important behavior:

- Track-specific player averages ignore scores above 15.
- Team track averages divide by `len(scorelist) / 5`, assuming 5 players per team.
- Overall player averages use `score * 12 / len(scorelist)`.
- Bagging players are represented by `extract.py` as `player_name + " (bag)"` when an individual race score is `1`.

Caching

`Flask-Caching` is configured with:

```
CACHE_TYPE = "simple"
CACHE_DEFAULT_TIMEOUT = 3600
```

Several endpoints cache by query string for one hour. This is useful for performance, but it means newly changed CSV data may not appear immediately in a running process unless the process restarts or the cache expires.

Known Backend Issues And Risks

- No upload endpoint exists.
- No automatic extraction/rebuild step is wired into Flask.
- `extract.py` writes CSVs but is not called by the API.
- API responses often return formatted strings, forcing frontend parsing in `BestMatchups`.
- `team` is not validated for some endpoints before calling helper functions.
- Errors are returned as HTTP 400 for nearly all exceptions.
- `findteamavg` assumes 5v5 format.
- Current CSV naming supports division, not season.
- Cache invalidation will matter once Season 3 updates are live.